P2I - xFG

Point d’avancement intermédiaire

Référence/version	Point d’avancement_xFG
Projet	xFG
Date de début	02/03/2026

Auteur
GARNIER Mathis

TABLE DES MATIÈRES

I. INTRODUCTION **3**
I.1 Rappel du projet et objectifs 3
II. Ce qui a été fait **4**
II.1 Formation au Machine Learning 4
II.2 Pipeline ETL — Acquisition et nettoyage [FR-DAT-01] 4
II.3 Feature engineering 5
II.4 Modélisation xFG% 7
II.5 Metrique POE [FR-DAT-03] 9
II.6 Organisation du code 10
III. Ce qu'il reste à faire et à améliorer **11**
III.1 Ameliorations du modele ML 11
III.2 Développement de l'API REST et du Frontend 11
III.3 Limite théorique du modèle 11
IV. Planning recalé **13**

I. INTRODUCTION

I.1 Rappel du projet et objectifs

Le projet xFG vise à faire ce qui est fait dans le football avec les Expected Goals (xG) mais ici adapté au basketball. Le FG% (Field Goal Percentage) classique est biaisé car il ne différencie pas un tir facile sous le panier d'un step-back à trois points.

Pour rappel nos trois objectifs majeurs étaient :

• [FR-DAT-02] Modéliser la métrique xFG% : probabilité de réussite de chaque tir via du machine learning.
• [FR-DAT-03] Calculer une nouvelle métrique, le POE (Points Over Expectation = Points réels - Points attendus) , pour voir réellement la qualité de tir d'un joueur est donc intrinsèquement son niveau de jeu.
• [FR-VIS-01/02/03] Restituer les résultats obtenus via un site web avec un demi-terrain interactif où l'on pourrait aussi voir le classement des joueurs du meilleur au moins bon.

Les technologies utilisées sont pour le Backend, du Python (ML + API REST JSON) et pour le Frontend, du HTML/CSS/JavaScript.

II. Ce qui a été fait

II.1 Formation au Machine Learning

Pour commencer le projet, n'ayant pas encore eu les cours du S8 sur le Machine Learning, j'ai dû apprendre à l'aide d'internet le fonctionnement du ML.

Pour ce faire, j'ai suivi pendant les deux premières semaines du projet les cours en ligne de Andrew Ng. Cela m'a permis de mieux comprendre les mécanismes et outils que les experts utilisés pour le Machine Learning que ce soit à travers le nettoyage des données en features ou les différents algorithmes qui existaient afin d'utiliser ceux qui correspondent le mieux à mon projet.
J'ai résolu ce problème de manière simple en prenant simplement un jeu de données public via Kaggle qui fourni exactement les mêmes données que l'API NBA donc je n'ai pas eu a modifié mon code.

II.2 Pipeline ETL — Acquisition et nettoyage [FR-DAT-01]

La première partie du projet était donc de récupérer un jeu de données. Dans un premier temps, j'avais comme idée d'utiliser l'API officiel de la NBA mais après de trop nombreuses requêtes, je me suis fait bannir mon IP et ne pouvait donc plus utiliser l'API.
J'ai pour le moment créé mon modèle seulement sur les tirs de la saison 2023-2024 soit 218 050 tirs après filtrage (je n'ai pas considéré les tirs de plus de 35 ft car souvent des tirs pris en fin de match ou de quart-temps et qui pouvaient fausser les résultats car non-représentatifs).
Notons aussi que certaines données ont été "nettoyés" dans le sens où un booléen de type "True/False" était converti en binaire 1/0 car ne pouvant être lu par la bibliothèque "sklearn" utilisé ici.

II.3 Feature engineering

Cette partie concerne l'entraînement du modèle à l'aide du "feature engineering". A l'aide du cours de Andrew Ng, on nous apprend qu'il faut fournir à celui-ci des données qu'il peut exploiter. Un modèle ML ne "comprend" que des chiffres, il cherche des corrélations entre des colonnes numériques et le résultat (tir réussi ou raté). Si les données brutes expriment mal l'information pertinente, le modèle rate des patterns pourtant évidents. C'est le rôle du feature engineering qui consiste à transformer les données brutes en variables explicatives qui encodent des connaissances de base que le modèle ne peut pas connaître car spécifique au basket.

Dans ce travail, six nouvelles variables ont donc été construites. Chacune correspondant à une hypothèse concernant les facteurs susceptibles d'influencer la difficulté d'un tir.

- 1. SHOT_ANGLE

  Les données initiales contiennent les coordonnées du tir (LOC_X et LOC_Y_Y). Le modèle pourrait théoriquement apprendre la relation entre ces deux variables, mais on peut lui faciliter la tâche en calculant directement l'angle du tir avec la fonction arctan2(LOC_X, LOC_Y). Cette variable permet notamment de capturer certaines particularités du terrain, comme l'avantage géométrique des corner 3 : ces tirs sont plus proches du panier (22 pieds) que les tirs à trois points classiques (23,75 pieds). En revanche, ils présentent aussi un inconvénient qui est qu'il est impossible d'utiliser la planche.

- 2. IS_CREATION

  La variable ACTION_TYPE contient de nombreuses valeurs textuelles décrivant le type de tir. Comme un modèle ne peut pas interpréter directement ce texte, je l'ai transformée en une variable binaire (0 ou 1). Elle indique si le tir est pris en création, c'est-à-dire après dribble (Pullup, Step Back, Fadeaway, Turnaround, Driving), ou non. Cette transformation

intègre une connaissance du jeu car ces tirs sont généralement plus difficiles car utilisés quand un défenseur est proche.
Les données confirment cette hypothèse : on observe environ 8 points de pourcentage de différence de réussite (FG%) entre les tirs en création et les tirs catch-and-shoot.

- 3. IS_CORNER_3

  Cette variable identifie les tirs pris dans les corners à trois points grâce aux coordonnées (LOC_Y < 9.2 et |LOC_X| > 22).
  Ces tirs sont statistiquement plus efficaces car ils sont plus proches du panier que les tirs à trois points situés sur l'arc classique mais aussi car les joueurs sont souvent sans défenseurs dans ces zones et s'y entraînent donc aussi plus.

- 4. TIME_PRESSURE
  Cette variable représente le temps restant dans le quart-temps, calculé en secondes (MINS_LEFT × 60 + SECS_LEFT).
  Elle sert d'indicateur pour les tirs pris sous pression en fin de possession, lorsque le joueur est obligé de tirer rapidement avant la fin de la shot clock;

- 5. IS_CLUTCH

  Cette variable indique les situations de clutch, c'est-à-dire les moments décisifs : quatrième quart-temps ou prolongation avec moins de 60 secondes restantes.
  Ces tirs représentent environ 2,3 % des tentatives, mais ils se produisent dans des situations de forte pression pour les deux équipes.

- 6. SHOT_DISTANCE_SQ

  La relation entre la distance du tir et le taux de réussite n'est pas linéaire. Le pourcentage de réussite chute fortement entre 0 et 8 pieds, puis diminue plus lentement.
  Or, la régression logistique suppose par défaut une relation linéaire entre les variables et le résultat. Pour mieux modéliser cette courbe, j'ai ajouté le carré de la distance (distance^2). Cela permet au modèle de mieux représenter l'évolution réelle du taux de réussite.

II.4 Modélisation xFG%

Architecture et Prétraitement des données

La modélisation a été réalisée à l'aide de la bibliothèque scikit-learn. Pour structurer notre démarche, nous avons utilisé une Pipeline. Cet outil permet d'enchaîner automatiquement et de manière sécurisée deux étapes clés : le prétraitement des données, puis l'entraînement du modèle.
Le prétraitement est géré par la fonction ColumnTransformer, dont le rôle est d'appliquer un traitement spécifique selon le type de variable :

- Les variables numériques (comme la distance, l'angle ou la pression temporelle) subissent une standardisation pour être ramenées à une moyenne de 0 et un écart-type de 1. Cela évite qu'une variable avec de grandes valeurs absolues (ex: la distance en pieds) n'écrase le poids des autres variables lors de l'apprentissage.

- Les variables binaires (is_creation, is_corner_3, is_clutch) ne nécessitent pas de transformation et sont conservées telles quelles.

L'utilisation de la Pipeline est ici cruciale pour éviter le phénomène de data leakage (fuite de données). En effet, elle garantit que les paramètres de standardisation sont calculés uniquement sur le jeu d'entraînement (X_train), avant d'être appliqués au jeu de test. Si nous avions normalisé l'ensemble des données avant de les séparer, le modèle aurait indirectement "vu" des informations du jeu de test, rendant ainsi son évaluation artificiellement optimiste.

Choix des algorithmes

Pour la modélisation, deux algorithmes différents ont été implémentés et comparés. Dans un premier temps, une Régression Logistique a été utilisée. Il s'agit d'un modèle linéaire simple, idéal pour valider notre pipeline de données, tester nos hypothèse et établir une première "vraie" ligne de référence au-delà du simple hasard.
Cependant, le basketball contient de nombreuses relations non-linéaires et complexes entre les variables. C'est pourquoi un second algorithme plus puissant a été mis en place l'XGBoost. Ce modèle est capable de capter ces relations complexes. C'est ce modèle XGBoost qui a été retenu comme l'algorithme final pour notre modèle.

Evaluation

Pour mesurer la fiabilité de notre modèle, nous utilisons deux métriques complémentaires :

- La Log Loss : C'est notre métrique principale. Elle évalue la précision absolue de la probabilité en la comparant au résultat réel (1 si le tir est marqué, 0 s'il est raté). Plus les probabilités prédites collent à la réalité, plus le score Log Loss est bas.

- Le ROC AUC (Area Under the Curve) : Contrairement au Log Loss, cette métrique évalue la capacité de discrimination du modèle, c'est-à-dire son aptitude à classer correctement les tirs. Concrètement, un score ROC AUC de 0.642 signifie que si l'on prend au hasard un tir réussi et un tir raté dans notre jeu de données, notre modèle a 64,2 % de chances d'avoir attribué un xFG% plus élevé au tir qui est effectivement rentré. Un score de 0.5 équivaut au hasard (notre baseline naïve), et 1.0 correspondrait à un modèle de prédiction parfait.

  Les résultats obtenus sur notre jeu de test sont les suivants :

Modele	**Log Loss**	**ROC AUC**	**Commentaire**
Baseline naive (hasard)	0.6931	0.500	Reference — modele nul
Régression Logistique	0.6601	0.632	Dépasse nettement la baseline
XGBoost (modele actuel)	**0.6524**	**0.642**	**Meilleur résultat actuel**

II.5 Metrique POE [FR-DAT-03]

Une fois le modèle entraîné et capable de prédire le xFG% de chaque tir, l'objectif est d'agréger ces probabilités individuelles pour évaluer le niveau global de chaque joueur. C'est ici qu'intervient le POE (Points Over Expectation), implémenté dans src/metrics.py.
Concrètement, la fonction compute_player_stats() récupère les probabilités prédites via pipeline.predict_proba() pour l'ensemble des tirs, puis les agrège par joueur avec un groupby('PLAYER_NAME').

Pour chaque joueur, on calcule :

- FG% : la moyenne réelle des tirs réussis sur la saison
- xFG% : la moyenne des probabilités prédites par le modèle sur ces mêmes tirs
- DIFF = FG% réel – xFG% prédit (exprimé en points de pourcentage)

Pour garantir la robustesse statistique, un filtre est appliqué en amont. Seuls les joueurs ayant tenté au minimum 200 tirs sont inclus dans le classement. Ce seuil évite les anomalies statistiques, par exemple un joueur réussissant 3 tirs très difficiles sur 4 tentatives se retrouverait artificiellement en tête du classement sans que cela soit représentatif de son niveau réel.

II.6 Organisation du code

Le code a été refactorisé en modules Python séparés pour respecter le principe de séparation des responsabilités. Chaque fichier a un rôle unique, ce qui facilite la maintenance et l'évolution du projet :

- src/config.py : Regroupe toutes les constantes du projet (liste des features, hyperparamètres XGBoost, chemins des fichiers…).
- src/data_loader.py : Gère le chargement et le nettoyage initial des données (Pipeline ETL).
- src/features.py : Contient la logique de feature engineering pour construire nos 6 variables explicatives.
- src/model.py : Définit la pipeline ML, les fonctions d'évaluation et la validation croisée.
- src/metrics.py : Calcule la métrique métier finale (le POE) pour chaque joueur qualifié.
- src/visualization.py : Génère les graphiques d'analyse (comme la comparaison des courbes Log Loss).

- main.py : Le point d'entrée central qui orchestre l'exécution séquentielle de toutes les étapes.

III. Ce qu'il reste à faire et à améliorer

III.1 Ameliorations du modele ML

Pour ce qu'il reste à faire côté Machine Learning, j'aimerai segmenté le modèle en trois sous-modèle chacun spécialisé dans une zone du terrain.
L'un serait spécialisé derrière la ligne des trois points, un autre sur la partie mid-range et l'autre sur les tirs à moins de 5 pieds du cercle.

Il me reste aussi à réaliser une analyse SHAP qui permettrait de comprendre l'impact de chaque métrique sur la prédiction d'un tir. Par exemple, comprendre l'impact de la distance sur le fait qu'un tir rentre ou non en comparaison à l'impact de la pression dû au temps restant sur la pendule avant un tir.

III.2 Développement de l'API REST et du Frontend

Le backend ML est désormais opérationnel. Les prochaines étapes concernent la mise en place de l'architecture Client-Serveur définie dans le cahier des charges.
Il s'agira tout d'abord de créer l'API REST via FastAPI afin d'exposer les résultats de la xFG% et du POE au format JSON.
Le développement se concentrera ensuite sur la partie visuelle avec la création du demi-terrain NBA interactif en HTML, CSS et JavaScript. Celui-ci intégrera des zones cliquables permettant de filtrer les statistiques par zone de tir, ainsi qu'un leaderboard dynamique affichant le classement des joueurs selon leur xFG% et leur POE.
Enfin, l'ensemble de ces interfaces devra être optimisé pour garantir un temps de réponse strictement inférieur à 2 secondes après chaque interaction de l'utilisateur.

III.3 Limite théorique du modèle

Même avec toutes les améliorations prévues, le modèle ne pourra probablement pas descendre sous un Log Loss d'environ 0.620. Cette limite s'explique par le bruit irréductible inhérent au basketball étant donné qu"une même situation de jeu ne produit pas systématiquement le même résultat.

La donnée manquante la plus impactante est la distance du défenseur le plus proche car uniquement disponible publiquement entre 2013 et 2016. La variable crée ACTION_TYPE tente de combler ce manque mais ne reflète sans doute pas la réalité.

IV. Planning recalé

Phase	Dates CdC	Dates reelles	Statut	Commentaire
Apprentissage Machine Learning + Benchmark / Etat de l'art	16/01-31/01	16/01-12/02	FAIT	Décalage de deux semaine dû à l'apprentissage du machine learning
Dev. Pipeline ETL	01/02-24/02	12/02-20/02	FAIT	Retard : blocage API NBA → basculement sur Kaggle
Modelisation xFG% & API	25/02-06/03	20/02-17/03	EN COURS	ML termine API REST reste a faire
Dev. Web (HTML/CSS) + API	07/03-20/03	18/03-5/04	A FAIRE	Decalage ~2 semaines
Calcul POE & Peaufiner modele 21/03-31/03	21/03-31/03		Supprimé	POE calcule en CSV Optimisation en cours
Optimisation UX & Rendu final	01/04-22/04	5/04-22/04	A FAIRE	Calendrier a tenir